
Guía de Angular

Ventas-Frontend · demo-ira-car

Del nivel Junior al Senior: explicación completa del proyecto

Creación con Angular CLI 21 · Arquitectura standalone + SSR · Servicios, pipes, componentes, rutas, guards, directivas e interceptores · Backend Spring Boot - JWT · Tests con Vitest · Manejo de errores · Despliegue Docker / nginx / Netlify / Railway

Generado a partir del estado real del repositorio (rama ventas/erik, commit 60f2f00)

Índice de contenidos

Guía de Angular — Ventas-Frontend (demo-ira-car)

1. Resumen del proyecto
2. Creación del proyecto desde cero
3. Estructura de carpetas del proyecto
4. Archivos de entrada de la aplicación
5. Configuración del proyecto
6. Servicios (core/service)
7. Interfaces y enums (core/interfaces, core/enums)
8. Pipes (core/pipes)
9. Componentes reutilizables (core/components)
10. Rutas: guards, directivas e interceptor (core/routes)
11. Features (módulos de negocio)
12. Relación con el backend
13. Tests (Vitest + jsdom)
14. Manejo de errores y excepciones
15. Despliegue (Docker, nginx, Netlify, Railway)
16. Preguntas de repaso (junior -> senior)
17. Resumen de arquitectura en una frase

Guía de Angular — Ventas-Frontend (demo-ira-car)

Guía de estudio desde nivel Junior hasta Senior basada en el proyecto real **Ventas-Frontend** (ERP/Punto de Venta de ventas). Explica el proyecto completo: cómo se creó, cómo está construido, arquitectura, relación con el backend, librerías, servicios, pipes, componentes, rutas, guards, interceptores, directivas, tests, manejo de errores y despliegue.

1. Resumen del proyecto

Es el **frontend** de un sistema de punto de venta y ERP (cobro en caja, ventas, productos, usuarios con roles y permisos, compras a proveedores, reportes e historial). Consume un backend **Spring Boot** llamado **Ventas-Backend** (carpeta hermana) en `http://localhost:8081/api`.

1.1 Stack

Angular 21 en modo **standalone** (sin NgModules), TypeScript ~5.9, RxJS 7.

SSR habilitado (`app.config.server.ts` + `app.routes.server.ts`) con **Prerender**.

Angular Material (`@angular/cdk`, `@angular/material`, tema Material propio en `src/material-theme.scss`).

jsPDF + jspdf-autotable para generar tickets y PDFs.

Chart.js 4 para gráficas de reportes.

xlsx para exportar a Excel (100% client-side).

zone.js 0.16 reactivado como motor de detección de cambios (Angular 21 es *zoneless* por defecto).

Vitest + jsdom para unit tests (builder `ng test`).

Build con Angular CLI 21: `ng serve` en el puerto 4200, sin proxy.

Nombre del paquete npm: `demo-ira-car`.

1.2 Rutas de la app (app.routes.ts)

Ruta	Componente	Permiso requerido
---	---	---
`` (vacío)	Login	—
<code>/cobro</code>	Cobro (POS)	—
<code>/perfil</code>	Perfil	—
<code>/compras</code>	Compras (3 pestañas)	<code>VER_COMPRAS</code>
<code>/productos</code>	Productos (CRUD)	<code>VER_PRODUCTOS</code>
<code>/productos-desactivados</code>	Productos dados de baja	<code>VER_PRODUCTOS</code>
<code>/reportes</code>	Reportes	<code>VER_REPORTES</code>
<code>/usuarios</code>	Usuarios (CRUD)	<code>VER_USUARIOS</code>
<code>/usuarios-desactivados</code>	Usuarios dados de baja	<code>VER_USUARIOS</code>

Ruta	Componente	Permiso requerido
/salehistory	Historial de ventas	VER_VENTAS
/categorias	Categorías (CRUD)	VER_CATEGORIAS
/roles	Roles (CRUD)	VER_ROLES
/forgot-password	Olvidé contraseña	—
/reset-password	Restablecer contraseña	—
**	Redirige a /	—

Todas las rutas usan **lazy loading** con `loadComponent()`.

2. Creación del proyecto desde cero

2.1 Comandos base (cómo nació el proyecto)

```
s
h
```

Notas importantes de Angular 21:

`--ssr` añade `app.config.server.ts`, `app.routes.server.ts` y el render en servidor (Aquí con `RenderMode.Prerender`).

`--standalone` ya no se escribe porque es el comportamiento por defecto desde Angular 17/18 (aquí se usa un solo `App` raíz y cada feature es un componente standalone).

Se usa `zone.js` como `polyfill` y `provideZoneChangeDetection()` para volver a la detección de cambios clásica (el 2026-09-10 se identificó que en modo `zone/less` las respuestas HTTP no redibujaban las tablas).

2.2 Generación de código Angular (scaffolding)

```
s
h
```

2.3 Explicación de las banderas

Flag	Efecto
---	---
<code>--skip-tests</code>	No crea el archivo <code>.spec.ts</code> . Aquí el proyecto configuró tests a mano (o los agregó después).
<code>--standalone</code>	Crea el componente sin <code>NgModule</code> (standalone, con <code>imports:</code> en el decorador).
<code>--flat=false</code>	Crea el archivo dentro de una carpeta hija con su nombre (estructura por feature para no ensuciar la raíz).
<code>--style css</code>	Usa <code>css</code> como laneja de estilos.
<code>--ssr</code>	Prepara la app para renderizado del lado del servidor / prerender.

2.4 Comandos diarios de ejecución

```
s  
h
```

3. Estructura de carpetas del proyecto

Principio de organización: `core` = código reutilizable sin lógica de negocio particular; `features` = una carpeta por vista/modulo funcional (login, cobro, productos...). Cada feature importa lo que necesita de `core`.

4. Archivos de entrada de la aplicación

4.1 main.ts — punto de arranque

```
t  
s
```

`bootstrapApplication` reemplaza al viejo `platformBrowserDynamic().bootstrapModule(AppModule)`.

Se arranca directo el **componente raíz** `App` con su **configuración** `appConfig` (sin `NgModule`).

4.2 app.ts — componente raíz

```
t  
s
```

Es un componente **standalone**: el decorador `@Component` usa `imports`: en vez de declarar un `NgModule`.

`signal()` es la *reactividad moderna de Angular*. En aplicaciones zoneless se usa con `computed()/effect()`. Aquí la app corre con Zone.js (clásico), pero el raíz usa signals.

4.3 app.config.ts — providers globales

```
t  
s
```

En Angular moderno la configuración global se declara **en el provider array**, no en un `NgModule`.

`withInterceptors(...)` registra interceptores como **funciones** (`HttpInterceptorFn`), el nuevo estilo de Angular.

4.4 app.routes.ts — rutas con lazy loading

```
t  
s
```

`loadComponent()` carga la vista **bajo demanda** (code-splitting por ruta).

`canActivate: [PermissionGuard] + data: {permission}` protege la ruta según el permiso del usuario.

** es el wildcard: rutas no encontradas redirigen a login.

4.5 SSR: app.config.server.ts y app.routes.server.ts

```
t  
s
```

```
t  
s
```

`RenderMode.Prerender`: la ruta se **pre-renderiza a HTML estático** en build (mejora SEO y primer render).

Esto exige que el código no dependa de `window/localStorage` en tiempo de build -> por eso existen los helpers SSR-safe (`storage-utils`).

5. Configuración del proyecto

5.1 angular.json — aspectos clave

```
j  
s  
o  
n  
c
```

Conceptos que debes dominar en una entrevista:

fileReplacements: cambiar un archivo de código según el entorno en tiempo de build. Gracias a esto `ng serve` usa `localhost:8081` y `ng build` usa la URL de Railway.

Budgets: límites de tamaño del bundle inicial y de los estilos por componente. Si se exceden, el build falla.

outputHashing: all: añade hash a los archivos => cache-safe (los archivos cambian de nombre si cambia su contenido).

****assets** con `input: public, output: /**`: copia `public/_redirects` a la raíz del build (necesario para el fallback SPA de Netlify).

5.2 tsconfig.json — TypeScript estricto

```
j  
s  
o  
n  
c
```

``tsconfig.app.json`` excluye ``src/**/*.spec.ts``; ``tsconfig.spec.json`` incluye solo ``spec.ts`` y tipos de ``vitest/globals``.

5.3 Environments

```
t  
s
```

Patrón usado en todos los servicios: ``private api = ${environment.api}/auth;`` — la URL del backend se centraliza y cambia según el entorno.

6. Servicios (core/service)

6.1 ¿Qué es un servicio en Angular?

Una clase con lógica reutilizable (HTTP, estado, helpers) inyectable vía ****Inyección de Dependencias (DI)****. Con ``providedIn: 'root'`` se crea un ****singleton**** compartido por toda la app sin declararlo en ningún módulo.

```
t
s
```

6.2 ProductService — patrón CRUD con HttpClient

```
t
s
```

6.3 ReportService — parámetros opcionales con HttpParams

```
t
s
```

6.4 AuthService — seguridad de sesión local

```
t
s
```

Puntos entrevistables:

`try/catch` para tolerar `localStorage` manipulado (ej. JSON inválido).

Decodifica el payload JWT (`base64url`) solo para leer el `exp`; la firma la valida el backend.

`hasPermission(permission)` devuelve si el permiso está en el array del usuario de `localStorage`.

6.5 Servicios de estado local (sin HTTP)

CobroService: mantiene el carrito del POS en memoria con un `BehaviorSubject<CobroItem[]>` y valida stock sin llamar al backend.

TicketService: genera el ticket PDF con `jsPDF` (autotable), sin HTTP.

SidebarService: singleton que comparte `menuOpen` entre todos los componentes; se suscribe a `router.events` filtrando `NavigationStart` para cerrar el menú cuándo se navega (resuelve el bug de menús desincronizados).

```
t
s
```

7. Interfaces y enums (core/interfaces, core/enums)

Las interfaces son el **espejo de los DTO de Java** del backend: mismos nombres de campo que la respuesta JSON. Ejemplos:

```
t
s
```

```
t
s
```

```
t
s
```

Nota sobre `ProductForm`: tiene `hasHistory?: boolean` que decide en el template si el botón es **Eliminar** (borrado físico) o **Dar de baja** (borrado lógico). Ese dato lo calcula el backend.

8. Pipes (core/pipes)

8.1 ¿Qué es un pipe?

Un transformador de datos en el template: recibe un valor y devuelve otro formateado. Los pipes **puros** (por defecto) solo se recalculan cuando cambia la **referencia** del input, no en cada detección de cambios → eficiencia.

8.2 PaginatePipe — paginación client-side

```
t
s
```

Uso en el template (sin HTML, solo el contexto):

Detalle a comentar en entrevista: el **clamp** de `page` evita que al borrar el último registro de una página quede una tabla vacía.

9. Componentes reutilizables (core/components)

El proyecto muestra el patrón **presentación/controlador**: el componente padre es dueño del estado (`page`, `pageSize`) y el componente de presentación solo recibe inputs y emite outputs.

```
t
s
```

Uso:

10. Rutas: guards, directivas e interceptor (core/routes)

10.1 PermissionGuard (CanActivateFn) — control de acceso por ruta

```
t
s
```

En Angular 17+ los guards son **funciones** (`CanActivateFn`), no clases.

`inject()` permite usar DI dentro de funciones.

`route.data` es la metadata que se pone en `app.routes.ts` (`data: { permission: 'VER_PRODUCTOS' }`).

10.2 HasPermissionDirectives — directiva estructural *hasPermission

```
t
s
```

Uso en los templates:

Explicación de entrevista: las directivas estructurales usan `TemplateRef` (la plantilla que hay que mostrar o no) y `ViewContainerRef` (el contenedor donde se inserta o se limpia). El setter se re-ejecuta cada vez que cambia el valor del input.

10.3 AuthInterceptor (HttpInterceptorFn) — JWT + logout global en 401

```
t
s
```

Puntos fuertes para entrevista:

Se registra con `withInterceptors([...])` en `provideHttpClient`.

Clona la request (`req.clone`) en lugar de mutarla (inmutabilidad RxJS).

Maneja el **401 de forma global** (cierre de sesión automático) pero **excluye los endpoints de auth**, donde un 401 es una credencial incorrecta normal.

`throwError(() => err)` re-emite el error para que el componente que llamó pueda manejarlo también.

10.4 storage-utils.ts — SSR-safe

```
t
s
```

Por qué es clave: con ****SSR/prerender**** el código corre también en Node; tocar `'window'` sin guard lanzaría `'ReferenceError'` y rompería el build.

11. Features (módulos de negocio)

11.1 Login (features/login)

`AuthService.login()` recibe `LoginRequest` y devuelve `LoginResponse` (con `token` y `permissions`).

`saveToken()` / `saveUser()` guardan sesión vía helpers seguros.

`MatSnackBar` muestra feedback ("Bienvenido", errores de credenciales).

Tiene `ForgotPassword` y `ResetPassword` (flujo de recuperación de contraseña).

11.2 Cobro / POS (features/charge) — el núcleo

`cobro.ts` es la vista; delega toda la lógica a **facades**:

`facade/sale-facade.ts` (~511 líneas): carrito, búsqueda, barcode, métodos de pago, **polling de estado de pago cada 5s**, ticket PDF y venta en efectivo.

`facade/cash-facade.ts`: abrir/cerrar caja, resumen/corte y diferencia preliminar.

Patrón **facade**: el componente no habla con servicios directamente; pasan por una clase intermedia que orquesta (mejor testabilidad y menos lógica en el template).

Métodos de pago: efectivo, débito y crédito (**enum `PaymentMethod`**).

Reintento de pago con tarjeta: si el pago queda pendiente (`pendingPaymentId`), se puede reintentar (`retryCardPayment`).

`TicketService` genera el PDF del ticket con **jsPDF + autotable**.

El carrito vive en `CobroService` con `BehaviorSubject<CobroItem[]>`.

Permisos usados: ``ABRIR_CAJA``, ``CERRAR_CAJA``, ``CREAR_VENTAS``, etc.

11.3 Productos, Categorías, Usuarios, Roles (CRUDs)

Todos siguen el mismo esqueleto:

Tabla responsive (`.table-container` / `.table-card` / `.table-scroll`) + formulario de alta/edición al lado.

Paginación client-side con `PaginatePipe` + `PaginationControl`.

Filtros client-side con getters: `selectedCategory` en productos; por rol en usuarios.

Botones unificados: `.edit` (amarillo), `.delete` (rojo), `.cancel` (gris).

Productos usa **multipart** (`FormData`) para subir imagen con nombre/stock/precio.

Ejemplo clave de productos (borrado lógico vs físico):

```
t
s
```

``imageUrl()`` completa rutas relativas del backend contra ``environment.api``, para que las imágenes sigan funcionando cuando el backend está en Railway:

```
t
s
```

11.4 Vistas de "datos de baja"

deactivated-products: `GET /api/products/inactive`, botón "Dar de alta" (`PATCH /products/{id}/active`).

deactivated-users: filtra client-side los inactivos de `GET /api/users`.

Simétricas: tabla + paginación + botón de alta + menú sándwich.

11.5 Compras (features/shopping)

3 pestañas: Compras (historial + registro), Proveedores (CRUD), Márgenes.

Historial con renglones expandibles por compra y filtro por proveedor + paginación.

Modal "Nueva compra" con renglones (`LineaCompra[]`), subtotales, `compraValida` y permisos `CREAR_COMPRAS`, `CANCELAR_COMPRAS`, `EDITAR_PROVEEDORES`, `ELIMINAR_PROVEEDORES`, `VER_REPORTES` (márgenes).

Usa `ProviderService` (CRUD de proveedores), `PurchaseService` (crear/cancelar compras, revertir stock), `ProductService` (combo de productos) y `ReportService.getMargins()`.

11.6 Reportes (features/reports)

Chart.js 4: tendencia de ventas (line), métodos de pago (doughnut), top productos (bar), rendimiento por categoría (bar secundaria).

Tarjetas resumen (`GET /api/reports/summary`): ventas, total vendido, ticket promedio.

Stock bajo (`GET /api/reports/low-stock?threshold=`) y **corte de caja** (`GET /api/cash/summary`).

Exportaciones **100% client-side**: PDF con jsPDF/autoTable y Excel con `xlsx`.

Usa `PLATFORM_ID` para ejecutar Chart.js solo en el navegador (`isPlatformBrowser`), porque con SSR no existe `document` a la hora de preparar las gráficas.

11.7 Salehistory (features/salehistory)

Listado de `GET /api/sales`, paginado a 10 por página.

Filtros client-side: `selectedMethod`, `fromDate` y `toDate`. Compara ****strings de fecha yyyy-MM-dd**** (orden lexicográfico = cronológico, inmune a timezones).

Muestra resumen del filtro ("N ventas · Total: \$X") y `formatPaymentMethod()` para mostrar el método de pago.

12. Relación con el backend

Backend: **Spring Boot** (`Ventas-Backend`) en `http://localhost:8081/api`.

Peticiones directas (sin proxy). **CORS** del backend solo permite `http://localhost:4200`.

JWT: el `AuthInterceptor` agrega `Authorization: Bearer <token>` a cada request.

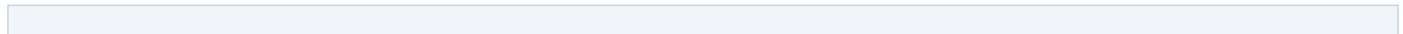
Token y usuario se guardan en `localStorage` (con helpers SSR-safe).

12.1 Endpoints principales

Servicio	Base	Uso
---	---	---
AuthService	<code>/auth</code>	login, forgot/reset/change-password, me/authorities
ProductService	<code>/products</code>	CRUD multipart, barcode, search, inactive, deactivate/activate

Servicio	Base	Uso
SaleService	/sales + /products	procesar venta, catálogo POS, barcode, search, historial
PaymentService	/payments	status/{tx} (polling), retry, reverse
CashService	/cash	active, open, close, summary
UserService	/users + /roles	CRUD usuarios, activar/desactivar, combo roles
RoleService	/roles	CRUD
PermissionService	/permissions	lista de permisos agrupados por módulo
CategoryService	/categories	CRUD (incluye updateCategory)
ProviderService	/providers	CRUD proveedores
PurchaseService	/purchases	listar/byProvider/crear/cancelar
ReportService	/reports	trend, top-products, payment-methods, categories, low-stock, margins, summary

Flujo típico de una petición segura:



13. Tests (Vitest + jsdom)

13.1 Configuración

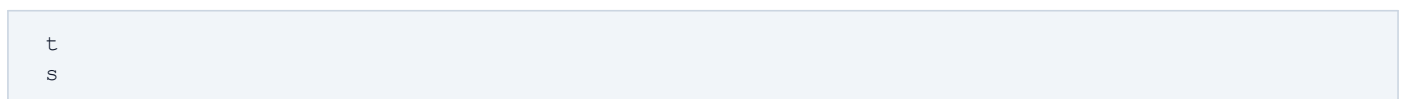
Builder: `@angular/build:unit-test` (angular.json test).

`tsconfig.spec.json`: `types: ["vitest/globals"]` (los describe/it/expect globales).

Dependencias dev: `vitest`, `jsdom`, `@types/node`.

Comando: `npm test (watch)` o `npx ng test --watch=false` (una vez).

13.2 Ejemplo real de spec



13.3 Detalles importantes aprendidos en el proyecto

Los componentes standalone se importan directo en `imports: []` del `TestBed` (sin `NgModule`).

****app.spec.ts****: la plantilla por defecto de `ng new` buscaba "Hello, Demo-IraCar"; se cambió a verificar el `<router-outlet>` y se agregó `provideRouter([])` como provider.

Los specs de login/forgot/reset fallaban con `NG0201: No provider found for ActivatedRoute` porque los templates usan `RouterLink`/inyectan `ActivatedRoute`. Se arregló agregando `provideRouter([])` en cada `TestBed`.

`fixture.whenStable()` evita que el `ngOnInit` dispare peticiones HTTP reales dentro de los tests (eso hacía que algunos specs "colgaran").

13.4 Cobertura actual

16 archivos de spec / 17 tests, todos pasan (verificado en esta rama `ventas/erik`).

Los features tienen un spec mínimo "should create" (prueba de humo); la lógica de filtros/paginación todavía no tiene tests unitarios dedicados.

14. Manejo de errores y excepciones

14.1 Errores de HTTP (HttpResponseError)

Patterns reales del proyecto:

```
t  
s
```

En un error HTTP se inspecciona ``err.status`` (401, 403, 404, 409...). Por ejemplo, el backend devuelve ****409**** cuando se intenta eliminar un producto con ventas asociadas → el callback muestra ``err.error?.message``.

14.2 Excepciones en TypeScript (try/catch)

```
t  
s
```

Regla de oro en entrevista: ****toda entrada externa (localStorage, JSON de API) debe tratarse como no confiable**** y parsearse dentro de try/catch cuando pueda romper el render.

14.3 Zoneless vs Zone.js (detección de cambios)

Angular 21 es **zoneless por defecto** (sin `zone.js`). Las respuestas de `HttpClient` no disparan detección de cambios automática.

Fix aplicado: volver a **Zone.js clásico** agregando `zone.js` a `polyfills` y `provideZoneChangeDetection()` en los providers. Desde entonces los callbacks HTTP redibujan las tablas al instante.

15. Despliegue (Docker, nginx, Netlify, Railway)

15.1 Dockerfile — build en 2 fases (multi-stage)

```
d  
o  
c  
k  
e  
r  
f  
i  
l  
e
```

Multi-stage: la imagen final solo tiene nginx + estáticos (más pequeña y segura). Se copia solo el bundle ``browser``, no el ``server`` (el SSR se sustituye por estático, igual que Netlify).

15.2 nginx.conf — fallback SPA + cache + headers

```
n
g
i
n
x
```

``try_files ... /index.html`` es el **SPA fallback**: al recargar ``/productos`` nginx devuelve ``index.html`` para que Angular resuelva la ruta (si no, daría 404).

15.3 .dockerignore

Excluye lo que no debe entrar al contexto de build de la imagen (reduce el tamaño y los secretos):

15.4 .gitignore

Excluye build (``/dist``), dependencias (``/node_modules``), configuraciones de IDE, coverage, ``.env`` y logs. **AGENTS.md** va subido a propósito (nota en el propio `.gitignore`: es la fuente de contexto del equipo para retomar trabajo en cualquier máquina).

15.5 Netlify + Railway

Build: `npx ng build -> dist/Demo-IraCar/browser.`

fileReplacements hace que en producción se use `environment-prod.ts` (HTTPS de Railway).

public/_redirects: `/* /index.html 200` (fallback SPA de Netlify; Netlify no usa nginx).

Railway (backend): la URL del API se reemplaza en `environment-prod.ts` por `https://TU-APP.up.railway.app/api.`

HTTPS obligatorio para que el navegador no bloquee peticiones mixtas (mixed content).

16. Preguntas de repaso (junior -> senior)

16.1 Nivel Junior

¿Qué es un componente standalone? Un componente que se declara a sí mismo con ``imports:`` en el decorador, sin necesidad de NgModule (default desde Angular 17+).

¿Qué es la Inyección de Dependencias? Un mecanismo del framework que construye y entrega dependencias (servicios) donde se declare en el constructor: ``constructor(private http: HttpClient) {}``. Con ``providedIn: 'root'`` el servicio es singleton.

¿Qué diferencia hay entre ``[valor]="x"`` y ``(evento)="y()"``? ``[]`` es property binding (de TS → template), ``( )`` es event binding (de template → TS).

¿Qué hacen ``*ngIf`` y ``*ngFor``? Dragón estructurales que condicionan el render o repiten un elemento por cada item (en este repo con el pipe ``paginate`` se pagina la lista).

¿Qué es un pipe puro? Un ``PipeTransform`` que solo se recalcula cuando cambia la referencia de sus inputs; los impuros se ejecutan en cada detección de cambios.

****¿Para qué sirve `async pipe`?**** Para desuscribir automáticamente de un Observable en el template. (Aquí no se usa: el proyecto se suscribe con callbacks.)

16.2 Nivel Medio

****¿Qué es un interceptor funcional y cómo se registra?**** `HttpInterceptorFn` que recibe `(req, next)` y devuelve un Observable. Se registra con `withInterceptors([...])` dentro de `provideHttpClient`.

****¿Por qué el proyecto reactivó Zone.js en Angular 21?**** Porque Angular 21 es zoneless y, en ese modo, las respuestas de `HttpClient` no disparan cambio de detección de cambios: las tablas no se redibujaban hasta un clic del usuario.

****¿Qué es un Guard CanActivateFn?** Una función que decide si una ruta puede activarse. Devuelve `true`, `false` o un `UrlTree`. Aquí valida el permiso del usuario (ej. `VER\_PRODUCTOS`) y redirige a `/cobro` si no lo tiene.

****¿Cómo funciona una directiva estructural personalizada?** Usa `TemplateRef` (plantilla a renderizar) y `ViewContainerRef` (contenedor). El setter `@Input()` decide si llama a `createEmbeddedView()` o `clear()`.

****¿Qué es lazy loading y cómo se indica en las rutas?** Cargar módulos/componentes bajo demanda con `loadComponent`/`loadChildren`. Puntos clave: menor `*initial bundle*` y division del bundle por rutas.

****¿Por qué `fileReplacements` en angular.json?** Para que el build cambie archivos según el entorno (dev → `environment.ts`; prod → `environment-prod.ts`).

****¿Qué es el modo Strict de TypeScript/Angular y qué aporta?** `strict: true` + `strictTemplates` → los templates se tipan, los inputs inaccesibles se detectan y menos errores en runtime.

16.3 Nivel Senior

****¿Por qué paginación client-side en este proyecto y cuándo NO conviene?** Los servicios traen toda la lista (catálogos pequeños de un POS) y el pipe trocea en el cliente. NO conviene con datos grandes: ahí se pasa a paginación server-side (`?page=&size;=`) o virtual scroll.

****¿SSR + Prerender con static hosting: qué limitación introduce y cómo se mitiga?** Con `RenderMode.Prerender` el código corre en Node al compilar → no existe `window`/`localStorage`. Se mitiga con helpers SSR-safe (`safeGetItem/safeSetItem/safeRemoveItem`).

****¿Por qué se separa la lógica del POS en facades (sale-facade, cash-facade)?** Para que el componente solo renderice y delegue; la lógica de negocio (carrito, polling de pago, ticket) queda aislada y testeable.

****¿Cómo atacarías los issues conocidos del proyecto?**

`alert()/confirm()` -> `MatSnackBar/MatDialog` (consistencia UX con el login).

Sin `takeUntil` en componentes -> `Subject + pipe(takeUntil(ngOnDestroy))`.

Warnings CommonJS (jsPDF/canvg/xlsx) -> `allowedCommonJsDependencies` en angular.json.

Bundles: `chart.js`, `xlsx` y `jsPDF` entran solo por lazy chunks de Reportes (documental el por qué el initial budget no revienta).

Paginación real en backend si las listas crecen (salehistory).

Autorización server-side de permisos: hoy el frontend lee `permissions` de `localStorage` (manipulable); el backend DEBE validar el JWT por endpoint.

Reemplazar la URL placeholder de `environment-prod.ts` al desplegar en Railway.

****¿Por qué el `AuthInterceptor` excluye `/auth/login` del manejo global de 401?**** Porque un 401 ahí no indica sesión expirada, sino ****credenciales incorrectas****: debe mostrar su snackbar de error en el formulario, no cerrar una sesión que no existe.

****¿Qué pasa si se despliega en HTTPS (Netlify) y el API sigue en HTTP?**** El navegador bloquea las peticiones mixtas (mixed content). Por eso `environment-prod.ts` exige HTTPS y se indica reemplazar la URL del API en Railway.

****¿Cómo se sube una imagen con Angular?**** Con `FormData`: `formData.append('image', file)` y `POST/PUT` con `HttpClient`. El backend la recibe como `MultipartFile`.

17. Resumen de arquitectura en una frase

Un ****frontend Angular 21 standalone con SSR****, organizado en `core` (reutilizable: servicios, pipes, componentes de UI, rutas/guards/interceptor) y `features` (una carpeta por vista), que se comunica con un ****backend Spring Boot**** vía REST + JWT a través de un interceptor, redirigiendo por permisos mediante un guard, paginando y filtrando en el cliente, y desplegándose como estáticos en Netlify o contenedores Docker/nginx.